Logisim IDE

Obsah

Úvod 2
Rozhranie 2
Prieskumník súborov 2
Nástroje 3
Pracovná časť 3
Integrovaný terminál 3
Pridávanie nových CPU 3
Kompailer 4
Funkcie 4
Vkladanie hodnôt do pamäte 5
Chybové kódy 5
0x001 5
0x002 5
0x003 5
0x011 5
0x021 5
0x022 5
0x031 5
0x032 5
0x033 5

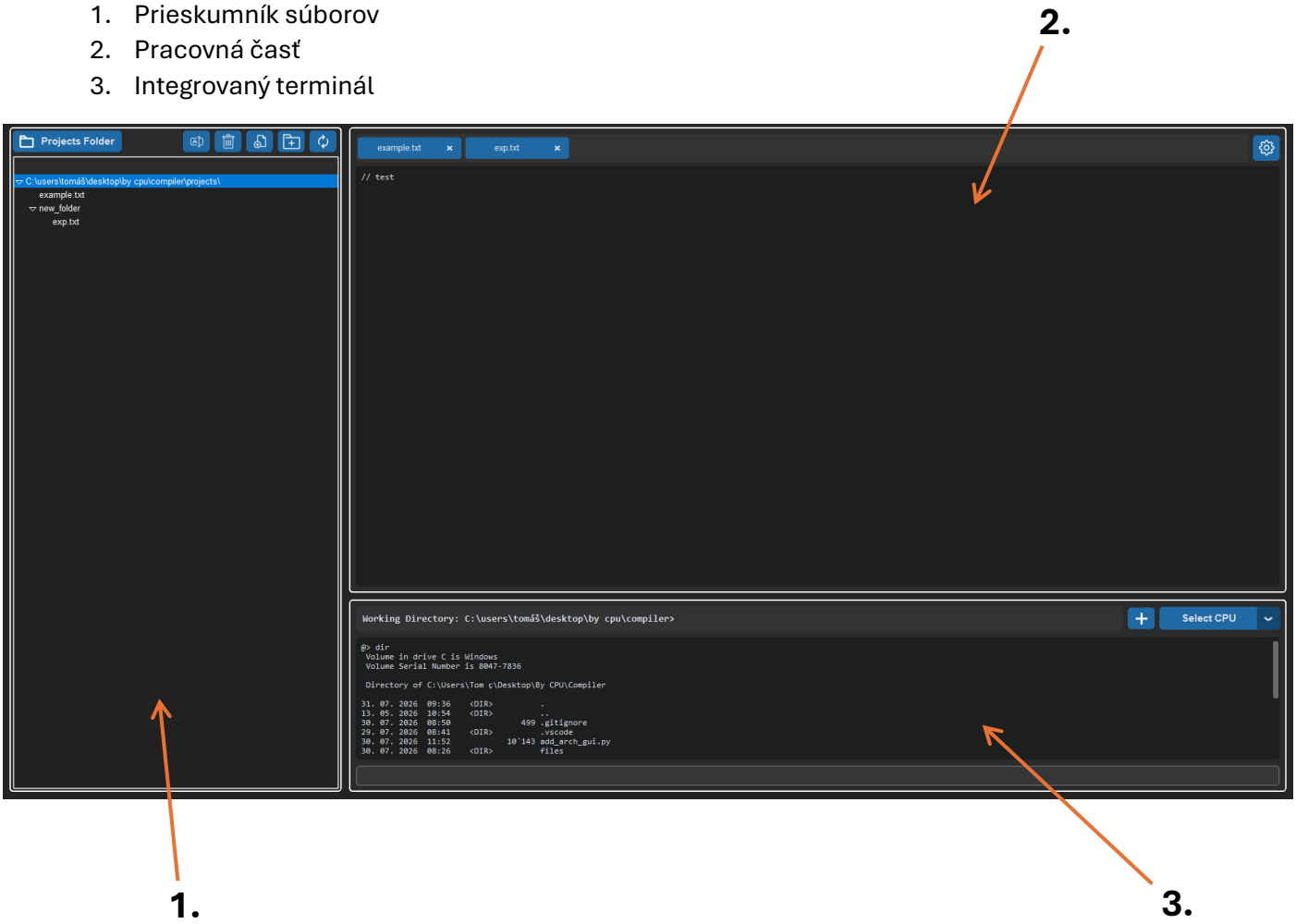
Úvod

Logisim IDE je program napísaný v pythone, určený na programovanie procesorov vytvorených v programe Logisim. Programovanie procesorov je prostredníctvom jazyku symbolických inštrukcií (ASM). Logisim IDE obsahuje Integrované Vývojárske Prostredie (IDE) ktoré obsahuje zabudovaný terminál a prieskumníka súborov, nástroj na pridávanie nových procesorov a vlastním virtuálnym kompailerom ktorý prepíše kód symbolických inštrukcií na binárne hodnoty (.bin) alebo na hexadecimálne hodnoty (.hex). Pre spustenie Logisim IDE je potrebné mať stiahnutý python interpreter a knižnicu „customtkinter“ (CTk).

Rozhranie

Rozhranie Logisim IDE pozostáva z troch hlavných častí:

1. Prieskumník súborov
2. Pracovná časť
3. Integrovaný terminál



Prieskumník súborov

Táto časť slúži na prácu so súbormi a priečinkami projektov. Nad samotným prieskumníkom sa nachádzajú nástroje pre manipuláciu a úpravu súborov.



Nástroje

Projects Folder – Otvára projektový priečinok vo Windows Explorere (C:\...\compiler\projects\)

– Premenovávanie zvoleného priečinka alebo súboru (F2)

– Odstraňovanie súborov alebo priečinkov (Del)

– Vytvoriť nový súbor

– Vytvoriť nový priečinok

– Obnovenie zabodovaného prieskumníka súborov (Ctrl + R)

Pracovná časť

Táto časť je určená na úpravu a ukladanie obsahu samotných súborov. Ukladanie je predvolené nastavené ako „auto_save = True“. V jeho vrchnej strane sa nachádza lišta pre aktuálne používané súbory a tlačidlo (Compile) ktoré skompiluje aktuálne otvorený súbor. V nižšej časti sa nachádza „Editor“ v ktorom sa priamo upravuje obsah súboru.

Integrovaný terminál

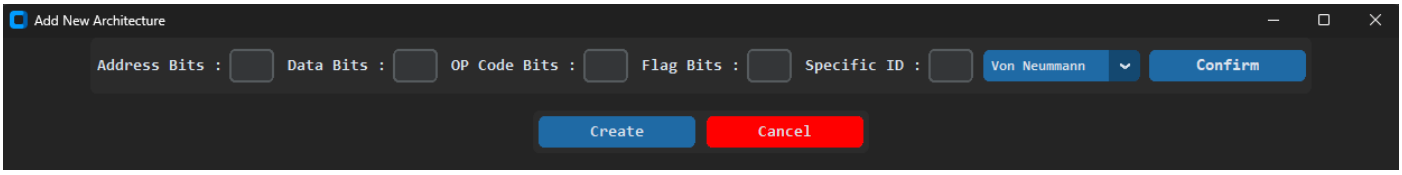
Integrovaný terminál je určený na zadávanie príkazov systému Windows a zobrazuje sa v ňom výstup kompilácie.



Vo vrchnej časti sa nachádza lišta s „Working Directory“, tlačidlo ktoré otvára nástroj na pridanie nového inštrukčného setu a parametrov procesoru a rozbaľovanie menu v ktorom volíte procesor pre ktorý chcete zvolený kód skompilovať. Pod oknom terminálu je zadávací riadok pre príkazy.

Pridávanie nových CPU

Nový procesor pridáte pomocou tlačidla . Následne sa otvorí nástroj pre pridávanie procesorov.



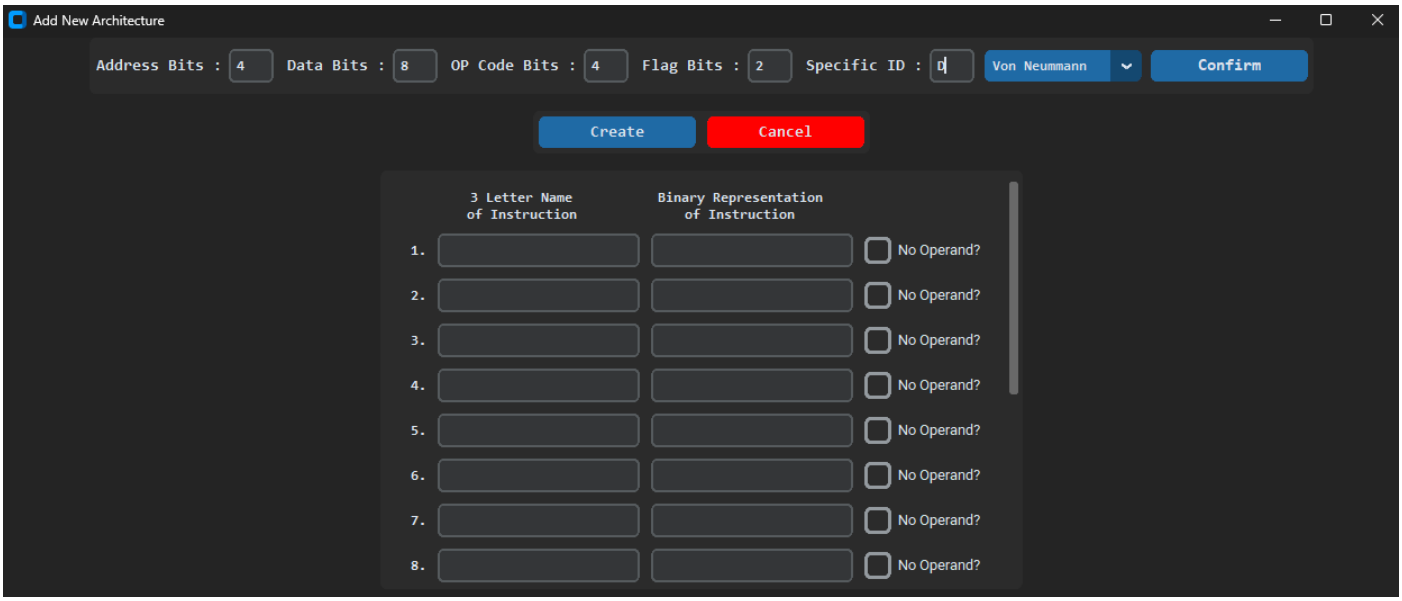
Povinné:

Následne zadáte počet adresných a dátových bitov, počet bitov pre inštrukciu a príznakov z ALU (flag) a vyberiete archytektúru.

Voliteľné:

Špecifický identifikátor – pokiaľ už CPU s rovnakým počtom bitov a architektúrou existuje treba ho rozpoznať cez špecifický identifikátor.

Následne potvrdíte parametre CPU a vygeneruje sa formulár pre zadávanie inštrukcií.



Následne zadáte názov inštrukcie v 3 písmenách a jeho binárnu reprezentáciu potom od kliknete či daná inštrukcia používa operand (hodnotu / adresu). Ak je označený „☑“ tak operand nepoužíva. Keď zadáte všetky inštrukcie kliknete na tlačidlo „Create“ tým sa vytvoria potrebné súbory pre kompiláciu.

Kompailer

Kompailer je priamo súčasťou celého programu avšak dá sa spustiť aj bez Logisim IDE pomocou príkazu:

„python C:\...\compiler\vcc.pyw --file [cesta k súboru] --cpu [názov procesoru podľa Logisim IDE]“

Kompailer nedopodporuje záporné čísla a nerozpoznáva indentácie a veľkosť písmen. Kompailer podporuje syntax „operácia hodnota;“ [ldi 12;]. V kóde je možné nechávať jednoriadkové komentáre pomocou „//“ ktoré bude kompailer ignorovať. Kompailer nepodporuje cykly!

Funkcie

V programoch je možné vytvárať funkcie (label) avšak pre správne fungovanie je potrebné aby procesor obsahoval register pre návratovú adresu ktorý umožní používanie funkcií alebo zásobník ktorý umožní používanie vnorených funkcií. Funkcia je definovaná nasledovne:

```
label:          // názov funkcie
        ldi 2;  // obsah funkcie
        str 4;  //      ||
        ldi 3;  //      ||
        add 4;  //      ||
        str 7;  //      ||
                //      \/
        ret;    // ===[koniec]===

label;          // volanie funkcie
                // po ukončení sa načíta adresa zo zásobniku / návratového registra
lda 7;          // načítanie výslednej hodnoty do registru A
out;            // načítanie výslednej hodnoty do výstupnáho registra

end;            // koniec programu
```

Vnorená funkcia znamená že je možné volať funkciu z funkcie avšak počet koľko krát možno toto spraviť závisí od kapacity zásobníku.

Premenné do funkcie možno vložiť len predpripraveným hodnôt na príslušné adresy odkiaľ si ich funkcia bude vedieť prevziať.

Vkladanie hodnôt do pamäte

Hodnoty do pamäte možno vložiť dvomi spôsobmi:

1. Cez operáciu „LDI“ – [ldi 2;] – inicializuje do registru A hodnotu 2 a jej následné uloženie do RAM pomocou operácie „STR“ – [str 4;].
2. Cez kompailerovú operáciu „VAL“, kompailer dokáže vložiť väčšie hodnoty do pamäte ak je možné cez operáciu „LDI“ tým že hodnotu vloží priamo do RAM, pričom je možná vložiť maximálnu hodnotu rovnej:

$$x = 2^d$$

d = počet dátových bitov.
x = hodnota.

Hodnoty vkladané cez operáciu „VAL“ sú vkladané na koniec RAM v poradí v akom sú definované.

```
// 4-bit adresa
VAL 81;         // hdonota 81 bude na adrese 14
VAL 73;         // hodnota 73 bude na adrese 15
```

Chybové kódy

0x001 – *InstructionSetArchitectureMismatchError:* Architektúrové údaje medzi súborom „instructions.json“ a „hardware_dependencies.json“ si neodpovedajú.

0x002 – *FileNotFoundError:* Bola zadaná nesprávna cesta k súboru obsahujúci kód.

0x003 – *CPUModuleNotFoundError:* Neboli nájdené kompilačné súbory pre konkrétny CPU.

0x011 – *OperandOutOfRangeError:* Zadaný operand je mimo platného rozsahu pre daný CPU.

0x021 – *SyntaxError:* Pokus o volanie nedefinovanej funkcie.

0x022 – *SyntaxError:* Na konci inštrukcie chýba , ; ‘.

0x031 – *MemoryOverflowError:* Kód vyžaduje viac pamäte ako disponuje CPU.

0x032 – *MemoryOverflowError:* Hodnota operácie „VAL“ prekročila kapacitu 1 Bytu.

0x033 – *MemoryOverflowError:* Bolo zadaných príliš veľa „VAL“ operácií, nedostatok pamäte na ich uloženie.